

# BEACONHOUSE NATIONAL UNIVERSITY



## LAB 11

**Name:** Saad Mughal

**BNU ID:** F2023-009

**Total Marks:** /20

**Due Date:** 6 May, 2025

**Lab Manual**

**For**

**COAL(SEC\_B)**

|                   |                 |
|-------------------|-----------------|
| Course Instructor | Ms. Nimra Abbas |
| Lab Instructor    | Ayesha Seher    |
| Semester          | Spring 2025     |

## **Task 1:** Double a Number using SHL

This task demonstrates how to double a number using the SHL instruction. SHL shifts bits to the left, effectively multiplying the value by 2.

### **Solution:**

```
org 100h
.model small
.stack 100h
.data
```

```
.code
```

```
start:
```

```
main proc
```

```
    ; TASK 1 - Double a number using SHL
```

```
    MOV AL, 2    ; AL = 00000010b (2)
```

```
    SHL AL, 1    ; AL = 00000100b (4) ? multiply by 2
```

```
    ; AL now contains 4
```

```
    mov ah, 4ch
```

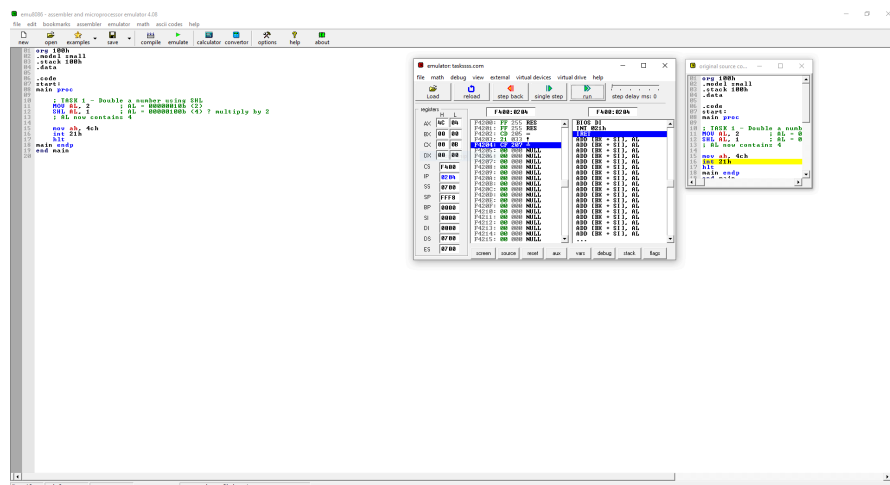
```
    int 21h
```

```
    hlt
```

```
main endp
```

```
end main
```

### **Screenshot:**



## **Task 2:** Apply SHL on Different Numbers

Use SHL to double various values and observe the pattern in results.

### **Solution:**

```
org 100h
.model small
.stack 100h
.data
```

```
.code
```

```
start:
```

```
main proc
```

```
    ; TASK 2 - Apply SHL on different numbers
    MOV AL, 5    ; AL = 00000101b (5)
    SHL AL, 1    ; AL = 00001010b (10) →  $5 \times 2$ 
    ; AL now contains 10
```

```
    mov ah, 4ch
```

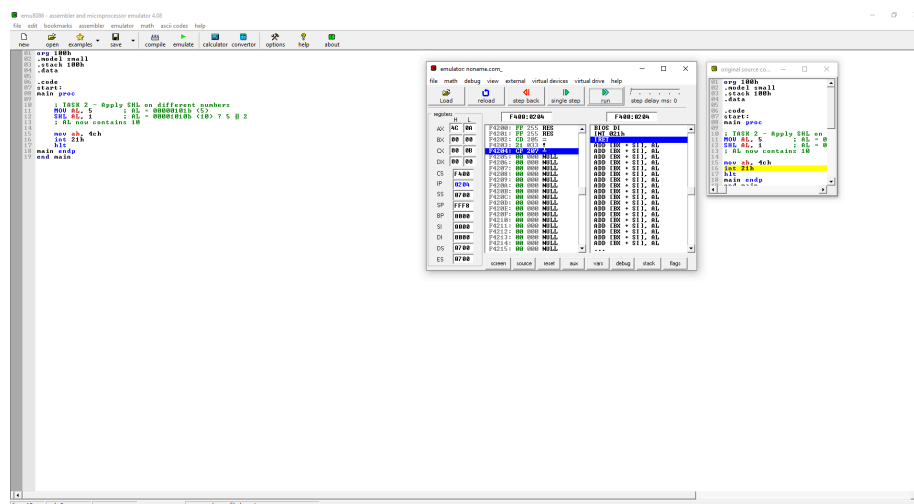
```
    int 21h
```

```
    hlt
```

```
main endp
```

```
end main
```

### **Screenshot:**



### Task 3: Halve a Number using SHR

SHR shifts bits to the right, dividing the number by 2 without keeping the remainder.

#### Solution:

```
org 100h
.model small
.stack 100h
.data
```

```
.code
```

```
start:
```

```
main proc
```

```
; TASK 3 - Halve a number using SHR
```

```
MOV AL, 9    ; AL = 00001001b (9)
```

```
SHR AL, 1    ; AL = 00000100b (4) → 9 / 2, discards remainder
```

```
; AL now contains 4
```

```
mov ah, 4ch
```

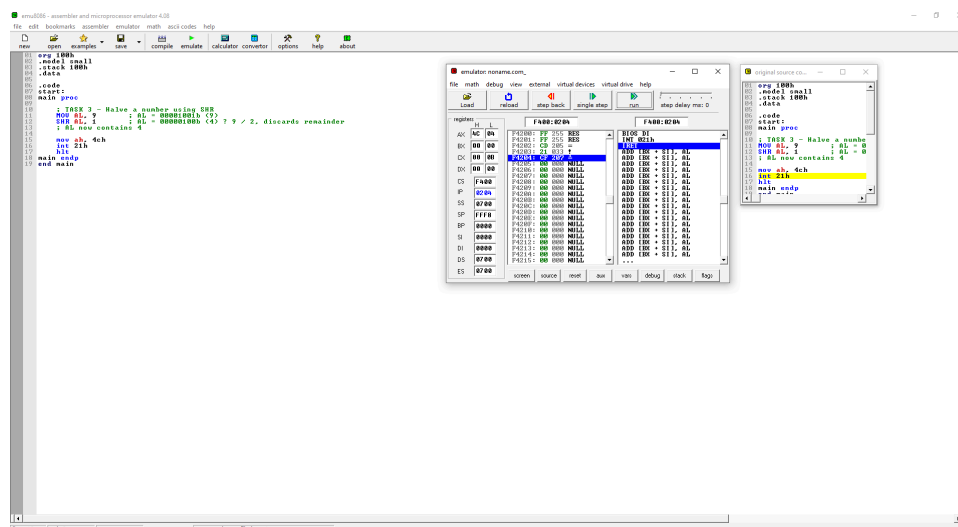
```
int 21h
```

```
hlt
```

```
main endp
```

```
end main
```

#### Screenshot:



**Task 4:** Task 4: Simulate Rotate Left using SHL and Carry Flag

If the ROL instruction is not available, simulate rotate by combining SHL and rotate carry manually (if covered in class).

**Solution:**

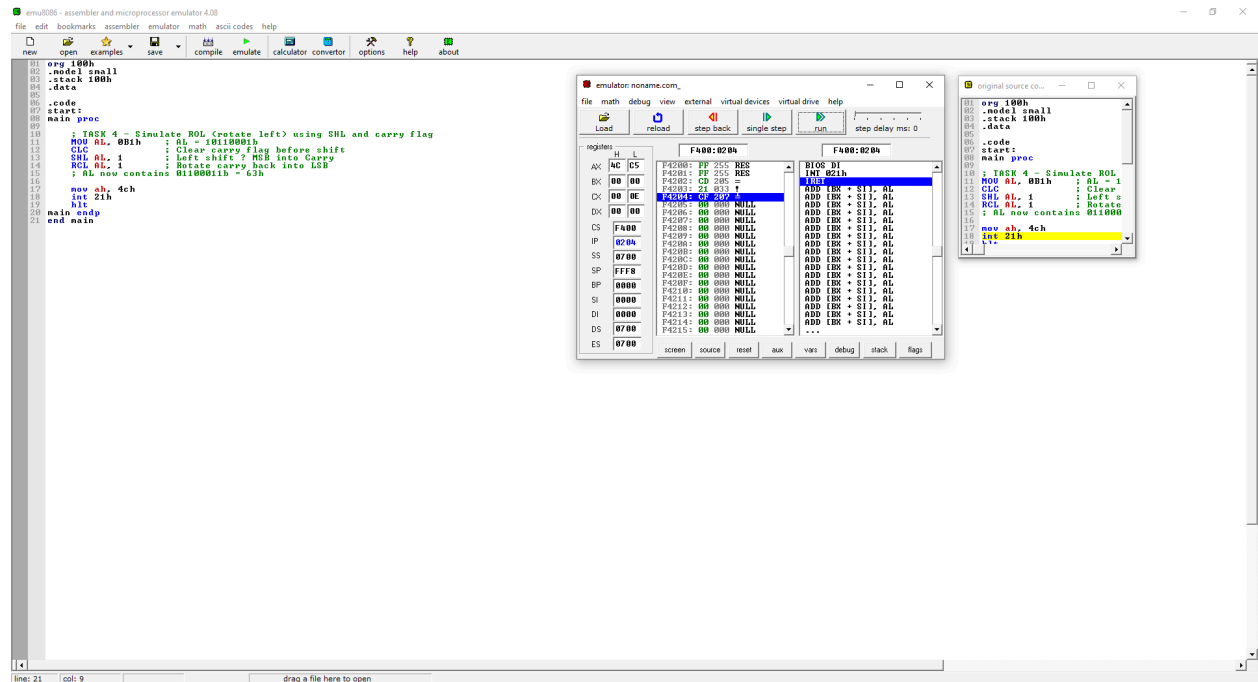
```
org 100h
.model small
.stack 100h
.data

.code
start:
main proc

    ; TASK 4 - Simulate ROL (rotate left) using SHL and carry flag
    MOV AL, 0B1h    ; AL = 10110001b
    CLC             ; Clear carry flag before shift
    SHL AL, 1       ; Left shift → MSB into Carry
    RCL AL, 1       ; Rotate carry back into LSB
    ; AL now contains 01100011b = 63h

    mov ah, 4ch
    int 21h
    hlt
main endp
end main
```

**Screenshot:**



## Task 5: Multiply by 3 using SHL and ADD

Combine SHL and ADD to multiply a number by 3.

### Solution:

```

org 100h
.model small
.stack 100h
.data

```

```

.code

```

```

start:

```

```

main proc

```

```

; TASK 5 - Multiply by 3 using SHL and ADD
MOV AL, 5 ; AL = 00000101b (5)
MOV BL, AL ; Store original value in BL
SHL AL, 1 ; AL = 00001010b (5 × 2 = 10)
ADD AL, BL ; AL = 00001111b (10 + 5 = 15)
; AL now contains 15

```

```

mov ah, 4ch

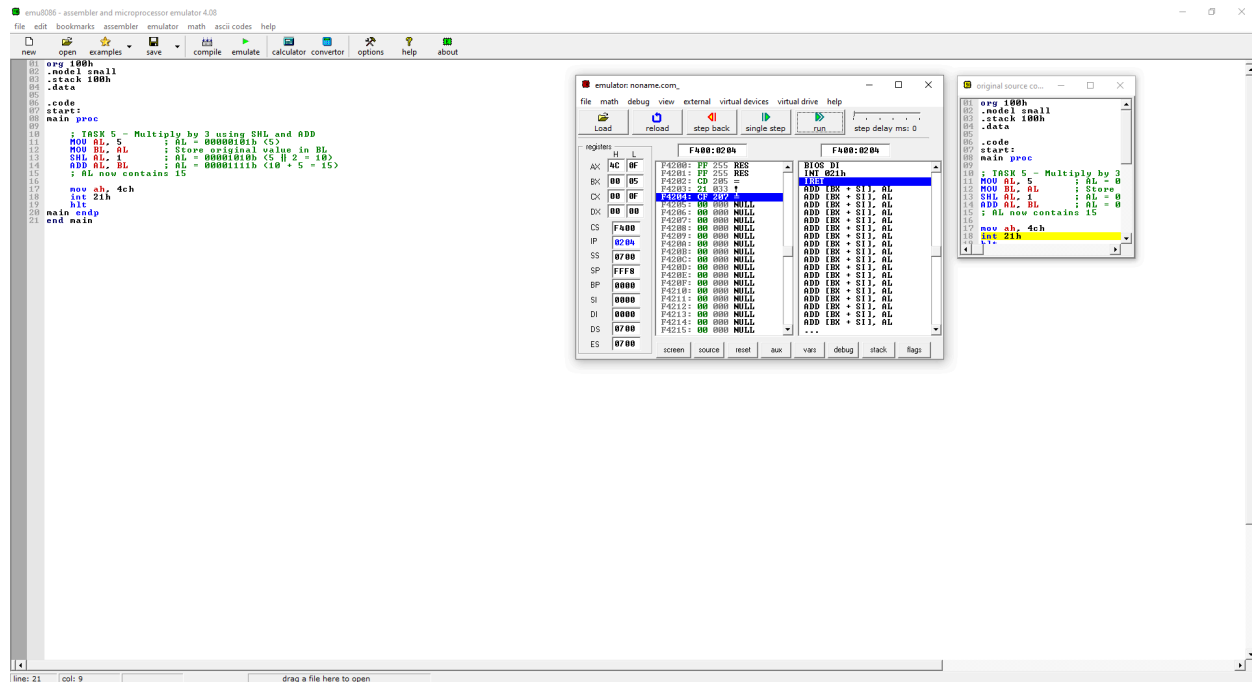
```

```

int 21h
hlt
main endp
end main

```

## Screenshot:



**Task 6:** Write an assembly program that doubles the value of a number. Remember that you cannot use the mul instruction for this task. The program should load a byte-sized value, double it and store the result back in memory.

## Solution:

```

org 100h
.model small
.stack 100h
.data
    number db 7    ; Initialize a byte-sized number (7)

.code
start:
main proc

    ; TASK 6 - Double the value without using MUL
    mov al, number ; Load the number into AL

```

shl al, 1 ; Double it using shift left

mov number, al ; Store the result back in memory

; Exit program

mov ah, 4ch

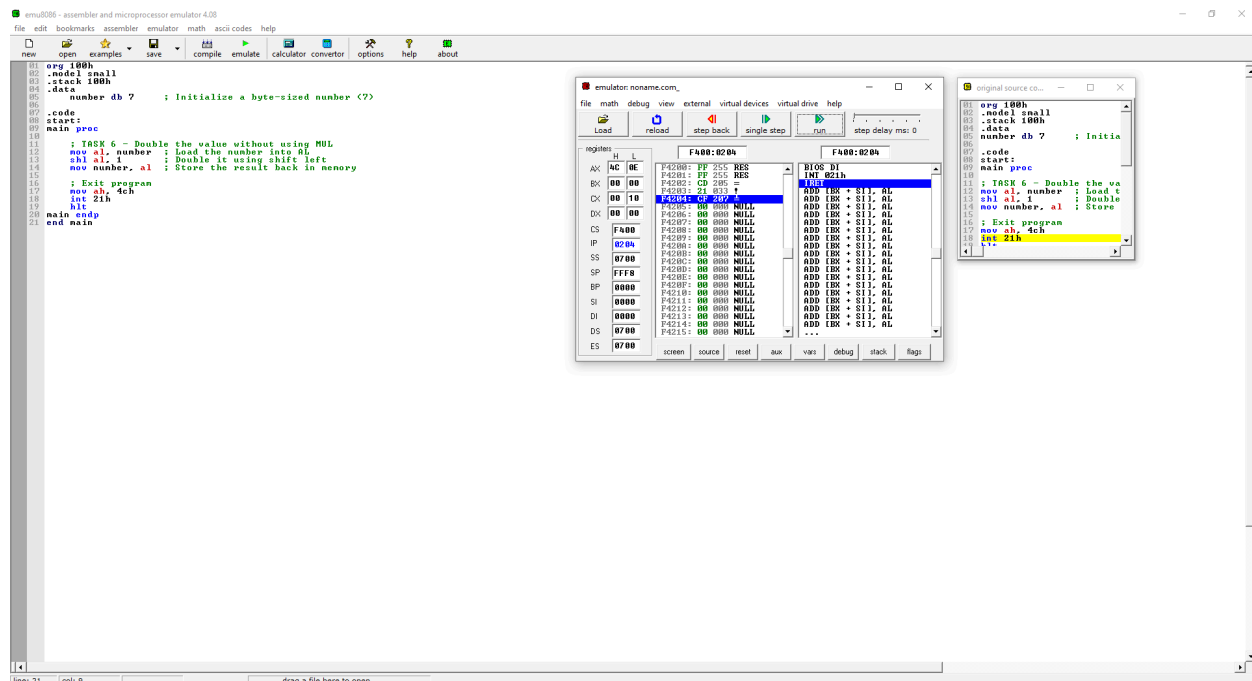
int 21h

hlt

main endp

end main

### Screenshot:



**Task 7:** Write an assembly program that performs a 4-bit multiplication using only bitwise shifts and additions. The program should handle two 4-bit numbers stored in an 8-bit space and compute their product without directly using the multiplication instruction.

### Solution:

org 100h

.model small

.stack 100h

.data

num1 db 5 ; First 4-bit number (0101b = 5)

num2 db 3 ; Second 4-bit number (0011b = 3)



result db 0 ; To store the result

.code

start:

main proc

; TASK 7 - Multiply two 4-bit numbers using SHL and ADD

xor ax, ax ; Clear AX

mov al, num2 ; Load multiplier (3)

mov bl, num1 ; Load multiplicand (5)

xor cl, cl ; Bit counter (0)

multiply\_loop:

test al, 1 ; Check if LSB of multiplier is 1

jz skip\_add ; If not, skip addition

add result, bl ; Add shifted multiplicand

skip\_add:

shl bl, 1 ; Shift multiplicand left (bl \*= 2)

shr al, 1 ; Shift multiplier right (next bit)

inc cl

cmp cl, 4 ; Repeat for 4 bits only

jne multiply\_loop

; Exit program

mov ah, 4ch

int 21h

hlt

main endp

end main

[Screenshot:](#)

```
001 org 100h
002 .model small
003 .stack 100h
004 .data
005     num1 db 5      ; First 4-bit number (0101b = 5)
006     num2 db 3      ; Second 4-bit number (0011b = 3)
007     result db 0     ; To store the result
008
009 .code
010 start:
011 main proc
012
013     ; TASK 7 - Multiply two 4-bit numbers using SHL and ADD
014     xor ax, ax       ; Clear AX
015     mov al, num2      ; Load multiplier (3)
016     mov bl, num1      ; Load multiplicand (5)
017     xor cl, cl        ; Bit counter (0)
018
019 multiply_loop:
020     test al, 1        ; Check if LSB of multiplier is 1
021     jz skip_add       ; If not, skip addition
022     add result, bl     ; Add shifted multiplicand
023     shl bl, 1         ; Shift multiplicand left (x2)
024     shr al, 1         ; Shift multiplier right (next bit)
025     inc cl            ; Repeat for 4 bits only
026     cmp cl, 4
027     jne multiply_loop
028
029 ; Exit program
030 mov ah, 4ch
031 int 21h
032
033 main endp
034 end main
```

emulator noname.com

file math debug view external virtual devices virtual drive help

Load reload step back single step run step delay ms: 0

| registers | H    | L    |
|-----------|------|------|
| AX        | 00   | 00   |
| CX        | 00   | 00   |
| DX        | 00   | 00   |
| SI        | 0000 | 0000 |
| DI        | 0000 | 0000 |
| BP        | 0000 | 0000 |
| SP        | FFFF | 0000 |
| IP        | 0000 | 0000 |
| CS        | 0000 | 0000 |
| DS        | 0000 | 0000 |
| ES        | 0000 | 0000 |

F400:0204

| Address   | Value | Comment |
|-----------|-------|---------|
| F400:0204 | 00    |         |
| F400:0205 | 00    |         |
| F400:0206 | 00    |         |
| F400:0207 | 00    |         |
| F400:0208 | 00    |         |
| F400:0209 | 00    |         |
| F400:020A | 00    |         |
| F400:020B | 00    |         |
| F400:020C | 00    |         |
| F400:020D | 00    |         |
| F400:020E | 00    |         |
| F400:020F | 00    |         |
| F400:0210 | 00    |         |
| F400:0211 | 00    |         |
| F400:0212 | 00    |         |
| F400:0213 | 00    |         |
| F400:0214 | 00    |         |
| F400:0215 | 00    |         |

screen source reset aux vars debug stack flags

original source code

```
001 mov bl, num1
002 xor cl, cl
003
004 multiply_loop:
005     test al, 1
006     jz skip_add
007     add result, bl
008     shl bl, 1
009     shr al, 1
010     inc cl
011     cmp cl, 4
012     jne multiply_loop
013
014 ; Exit program
015 mov ah, 4ch
016 int 21h
```